

1. Pattern Name

Façade Design Pattern

2. Category

Structural Design Pattern (GoF — Gang of Four)

3. Intent

The Façade Pattern provides a unified, higher-level interface to a set of interfaces in a subsystem. It defines a higher-level interface that makes complex subsystems easier to use by wrapping multi-step orchestrations, disparate dependencies, and fine-grained APIs into simple, cohesive operations for client callers, without restricting direct access to the subsystems.

4. Problem Statement

In real-world web backends, high-level business actions require orchestrating multiple independent modules. For example, a **User Registration Workflow** requires:

- Cryptographic hashing of plain-text passwords using `bcrypt` (Security Subsystem).
- Database connection and document insertion using raw `mongodb` driver (Persistence Subsystem).
- Cryptographic JSON Web Token (JWT) generation using `jsonwebtoken` (Auth Subsystem).
- SMTP transporter initialization and dispatch of transactional welcome emails via `nodemailer` (Notification Subsystem).

Without a Façade, the client controller/route handler must directly import, configure, instantiate, and coordinate all 4 libraries. This introduces massive coupling, violates the **Single Responsibility Principle (SRP)**, causes severe code duplication across routes/controllers, and makes maintenance fragile.

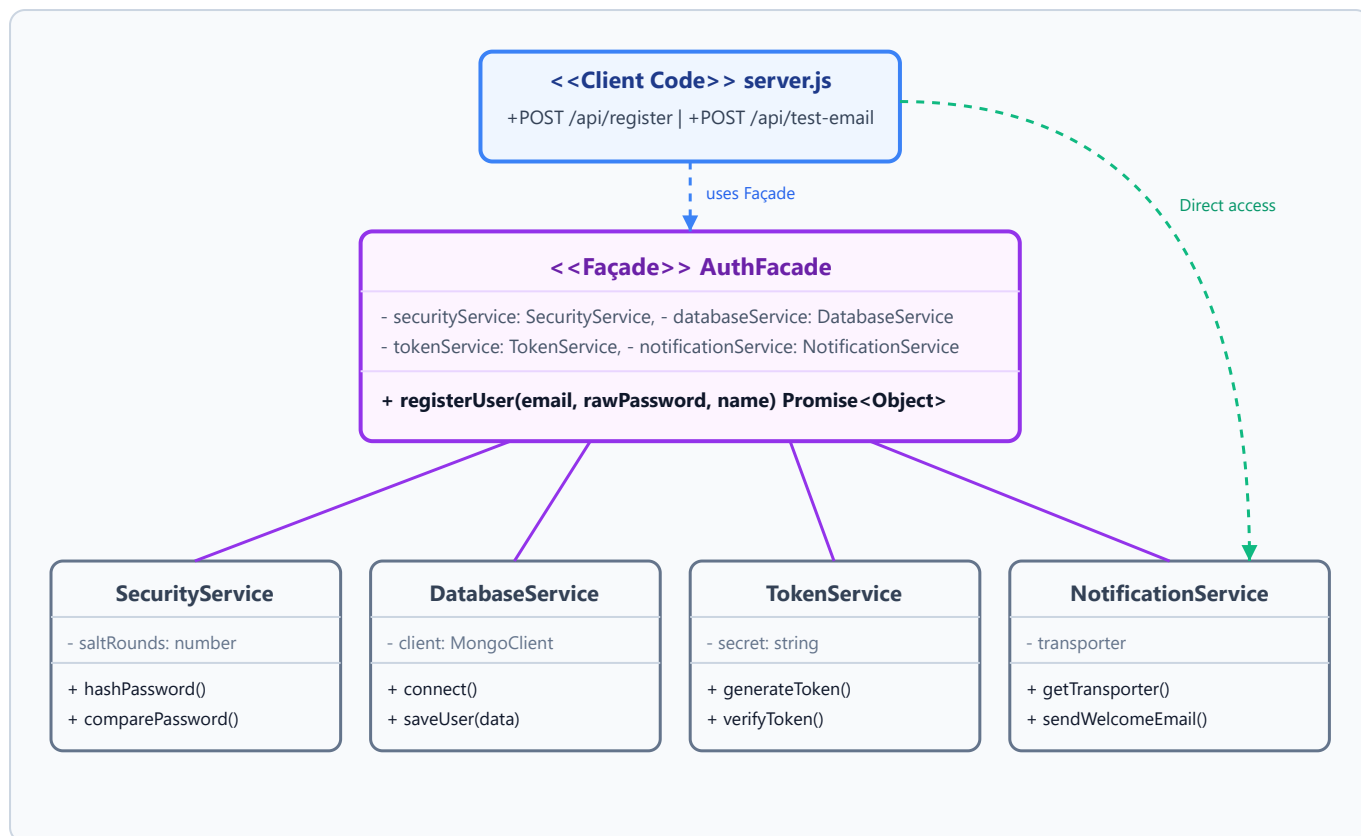
5. Motivation

Why is the standard direct implementation poor?

- **Tight Coupling:** The client code is tightly bound to low-level implementation details of four third-party libraries.
- **High Cognitive Load:** Client developers must know the internal execution order, parameter schemas, and error types of every subsystem.
- **Fragility:** Swapping the database driver (e.g., MongoDB to PostgreSQL) or mail provider forces modifications directly across all client routes.
- **Poor Reusability:** If user registration is invoked from multiple entry points (REST API, CLI script, background batch job), the entire 4-step sequence has to be copied and pasted.

The Façade pattern solves this by introducing `AuthFacade`, giving clients a single clean method call: `registerUser(email, password, name)`.

6. Pattern Structure (UML Class Diagram)



7. Class Responsibilities

Class / Participant	Role	Detailed Responsibilities
AuthFacade	Façade	Orchestrates the entire user registration pipeline. Holds composition handles to all 4 subsystem services; handles sequencing (Hash → DB → JWT → Email) and aggregates results into a clean structured payload.
SecurityService	Subsystem 1	Handles cryptographic operations using <code>bcrypt</code> . Generates salts, hashes raw passwords, and verifies password candidates.
DatabaseService	Subsystem 2	Connects to MongoDB using the plain official driver (<code>MongoClient</code>) and persists user documents into the database collection.
TokenService	Subsystem 3	Generates and verifies cryptographically signed JSON Web Tokens (JWT) using <code>jsonwebtoken</code> .
NotificationService	Subsystem 4	Configures Nodemailer with an Ethereum SMTP test account, composes HTML welcome messages, dispatches emails, and logs live web preview URLs.
server.js	Client	Defines HTTP API endpoints. Demonstrates clean, decoupled consumption via <code>POST /api/register</code> and standalone subsystem access via <code>POST /api/test-email</code> .

8. Code Implementation

The Façade Class (`facades/AuthFacade.js`):

```

class AuthFacade {
  constructor(securityService, databaseService, tokenService, notificationService) {
    this.securityService = securityService || new SecurityService();
    this.databaseService = databaseService || new DatabaseService();
    this.tokenService = tokenService || new TokenService();
    this.notificationService = notificationService || new NotificationService();
  }

  async registerUser(email, rawPassword, userName) {
    // 1. Security Subsystem: Hash raw password
    const hashedPassword = await this.securityService.hashPassword(rawPassword);

    // 2. Database Subsystem: Save document to MongoDB
    const savedUser = await this.databaseService.saveUser({ email, password: hashedPassword });

    // 3. Token Subsystem: Generate signed JWT
    const token = this.tokenService.generateToken({ userId: savedUser.id, email: savedUser.email });

    // 4. Notification Subsystem: Send welcome email
    const emailResult = await this.notificationService.sendWelcomeEmail(email, userName);

    return {
      success: true,
      data: { userId: savedUser.id, email: savedUser.email, token, emailDelivery: emailResult }
    };
  }
}

```

Client Route Integration (`server.js`):

```

// Extremely clean route decoupled from subsystem internals
app.post('/api/register', async (req, res) => {
  try {
    const { email, password, name } = req.body;
    const authFacade = new AuthFacade();
    const result = await authFacade.registerUser(email, password, name);
    return res.status(201).json(result);
  } catch (err) {
    return res.status(400).json({ success: false, error: err.message });
  }
});

```

9. Execution Flow (Object Interaction)

- Client Invocation:** HTTP client sends `POST /api/register` with JSON payload `{ email, password, name }`.
- Facade Entry:** The route handler delegates the payload directly to `AuthFacade.registerUser()`.
- Hashing Step:** `AuthFacade` invokes `SecurityService.hashPassword()` → salt generated and password hashed.
- Storage Step:** `AuthFacade` invokes `DatabaseService.saveUser()` → plain MongoDB client persists document and returns ID.
- Token Step:** `AuthFacade` invokes `TokenService.generateToken()` → signed JWT generated.
- Email Step:** `AuthFacade` invokes `NotificationService.sendWelcomeEmail()` → email dispatched through Ethereum SMTP with clickable preview link.
- Unified Response:** `AuthFacade` packages all results into a structured JSON payload and returns HTTP 201 to the client.

10. Advantages

- **Decoupled Architecture:** Shields client code from subsystem changes, library replacements, or API updates.
- **Reduced Code Duplication:** Multi-step registration orchestration is centralized in one place.
- **Clean Code & Maintainability:** Express route handlers remain short, readable, and focused strictly on HTTP concerns.
- **Subsystem Autonomy:** Underlying services remain completely independent and can be used on their own for fine-grained tasks.
- **Testability:** Subsystems can easily be mocked or stubbed via dependency injection in the Façade constructor.

11. Limitations

- **God Object Anti-Pattern Risk:** Over time, developers may continuously add unrelated operations to a single Façade, turning it into a bloated monolithic class.
- **Extra Abstraction Layer:** For simple single-step queries, adding a Façade adds unnecessary boilerplate and indirection.

12. Real-life Applications

- **E-Commerce Order Processing:** Coordinating inventory reduction, payment charging, invoice generation, packaging orders, and sending SMS confirmation.
- **Multimedia Transcoding Pipelines:** Demuxing media streams, video decoding, audio filtering, watermark insertion, and MP4 container packaging.
- **Cloud Infrastructure Provisioning:** Setting up VPC networks, subnets, firewall rules, compute instances, and DNS records through a single "createEnvironment" API.

13. Industry Examples

- **jQuery (\$.ajax):** Encapsulates low-level ActiveX, XMLHttpRequest, and CORS handling across different browser vendors.
- **Java Spring Framework (JdbcTemplate):** Serves as a Façade over raw JDBC connection handling, statement compilation, transaction commits, and rollbacks.
- **Google Cloud Client Libraries:** The Storage client library provides a simple upload() façade over chunked multipart HTTP requests, OAuth token refreshes, and exponential backoff retry algorithms.

14. Demonstration (Live Execution Steps)

1. **Start Server:** Run npm start in terminal → Express listens on http://localhost:3000 .
2. **Open Interactive Web UI:** Visit http://localhost:3000 in any web browser.
3. **Test Façade Registration:** Enter email & password → click "Register via Façade" → observe terminal log showing 4 delegated steps → observe response containing JWT and clickable Ethereum Email preview URL.
4. **Test Subsystem Independence:** Click "Send Direct Email" under POST /api/test-email → observe direct execution of NotificationService bypassing the Façade.

15. Conclusion

The **Façade Pattern** successfully solves subsystem orchestration complexity while preserving modular independence. By encapsulating Bcrypt, MongoDB, JWT, and Nodemailer operations within AuthFacade, we achieve clean separation of concerns, high testability, and decoupled client code in full accordance with the RUET CSE 3206 Software Engineering Sessional guidelines.